UCS 2312 Data Structures Lab
Assignment 8: Binary Heap and its applications

priorityQueueADT consists of integer element. Implement the following methods.
[CO2,K3]

- void insert(struct priorityQueueADT *P, int x) – Insertion new item into priority queue using Max Heap property
- int delete(struct priorityQueueADT *P) – Will remove the root of binary heap
- void display(struct priorityQueueADT *P) – Will display the contents pf Priority Queue

1. Demonstrate ADT with the following testcase

```
insert(p,14);
insert(p,16);
insert(p,22);
insert(p,11);
insert(p,9);
insert(p,18);
insert(p,10);
insert(p,7);
insert(p,4);
insert(p,1);
```

CODE:
maxheapadt.h

```
//max heap

struct pq
{
    int arr[1000];
    int size;

};

void init(struct pq *p)
{
    p->size=0;
    p->arr[0]=1000;
}

void insert(struct pq *p,int ele)
```

```
{
    int i;
    for (i=++p->size;p->arr[i/2]<ele;i=i/2)
    {
        p->arr[i]=p->arr[i/2];
    }
    p->arr[i]=ele;



}

void display(struct pq *p)
{
    for (int i = 1;i<=p->size;i++)
    {
        printf("%d ",p->arr[i]);
    }
}

int delete(struct pq *p)
{
    int i,child;
    int me,le;
    me=p->arr[1];
    le=p->arr[p->size--];

    for (i=1;(2*i)<=p->size;i=child)
    {
        child = i*2;
        if (p->arr[child+1]>p->arr[child])
            child++;
        if (le<p->arr[child])
            p->arr[i]=p->arr[child];
        else
            break;
    }

    p->arr[i]=le;
    return me;

}
```

Maxheap.c

```c
#include <stdio.h>
#include <stdlib.h>
#include "maxheapadt.h"

void main()
{
    printf("enter the number of elements:");
    int n;
    scanf("%d",&n);
    struct pq *p;
    p=(struct pq*)malloc(sizeof(struct pq));
    init(p);
    for (int i=0;i<n;i++)
    {
        printf("enter the element: ");
        int a;
        scanf("%d",&a);
        insert(p,a);

    }
    printf("\n---PRINTING--\n");
    display(p);
    printf("\n");
    printf("--1ST DELETION--\n");
    printf("MAX ELEMENT : %d",delete(p));
    printf("\n---PRINTING--\n");
    display(p);
    printf("\n");
    printf("--2ND DELETION--\n");
    printf("MAX ELEMENT : %d",delete(p));
    printf("\n---PRINTING--\n");
    display(p);
    printf("\n");
    printf("--3RD DELETION--\n");
    printf("MAX ELEMENT : %d",delete(p));
    printf("\n---PRINTING--\n");
    display(p);
    printf("\n");



}
```

OUTPUT:

```
enter the number of elements:10
enter the element: 14
enter the element: 16
enter the element: 22
enter the element: 11
enter the element: 9
enter the element: 18
enter the element: 10
enter the element: 7
enter the element: 4
enter the element: 1

---PRINTING--
22 14 18 11 9 16 10 7 4 1
--1ST DELETION--
MAX ELEMENT : 22
---PRINTING--
18 14 16 11 9 1 10 7 4
--2ND DELETION--
MAX ELEMENT : 18
---PRINTING--
16 14 10 11 9 1 4 7
--3RD DELETION--
MAX ELEMENT : 16
---PRINTING--
14 11 10 7 9 1 4
```

QUESTION 2. Write an application to design a priority queue using max binary heap. An item in the priority queue consists of employee id and salary amount. The queue supports two operations, namely, insertion and deletion.

Test the application with the following

```
insert(p,('A',15000));
insert(p,('K',12000));
insert(p,('R',4000));
insert(p,('T',3500));
insert(p,('L',4600));
insert(p,('P',6000));
insert(p,('Y',8600));
```

Output:
Employees are removed in the following order

('A',15000), ('K',12000), ('Y',8600), ('P',6000), ('L',4600), ('R',4000), ('T',3500),

ALGORITHM:

1.FUNCTION NAME: insert

INPUT:

i)struct pq *
ii)struct employee *

OUTPUT: None

```
1. int i;
2. for (i=++p->size;p->arr[i/2].salary<e->salary;i=i/2)
   {
        p->arr[i].id=p->arr[i/2].id;
        p->arr[i].salary=p->arr[i/2].salary;
   }
3. p->arr[i].id=e->id;
4.  p->arr[i].salary=e->salary;
```

2.FUNCTION NAME: delete

INPUT:

i)struct pq *

OUTPUT: char

```
1. int i,child;
2. char mec,lec;
3. mec=p->arr[1].id;
```

4. lec=p->arr[p->size].id;
5. long int les;
6. les=p->arr[p->size--].salary;

7. for (i=1;(2*i)<=p->size;i=child)
   {
   child = i*2;
   if (p->arr[child+1].salary>p->arr[child].salary)
   child++;
   if (les<p->arr[child].salary)
   { p->arr[i].id=p->arr[child].id;
   p->arr[i].salary=p->arr[child].salary;
   }
   else
   break;
   }
8. p->arr[i].salary=les;
9. p->arr[i].id=lec;
10. return mec;

CODE:

Pqadt.h

```
struct employee
{
    char id;
    long int salary;
};

struct pq
{
    struct employee arr[100];
    int size;

};

void init(struct pq *p)
{
    p->size=0;
    p->arr[0].salary=1000000;
}
```

```c
void insert(struct pq *p,struct employee *e)
{
    int i;
    for (i=++p->size;p->arr[i/2].salary<e->salary;i=i/2)
    {
        p->arr[i].id=p->arr[i/2].id;
        p->arr[i].salary=p->arr[i/2].salary;
    }
    p->arr[i].id=e->id;
    p->arr[i].salary=e->salary;

}

void display(struct pq *p)
{
    for (int i = 1;i<=p->size;i++)
    {
        printf("(%c,%ld) ",p->arr[i].id,p->arr[i].salary );
    }
}

char delete(struct pq *p)
{
    int i,child;
    char mec,lec;
    mec=p->arr[1].id;
    lec=p->arr[p->size].id;
    long int les;
    les=p->arr[p->size--].salary;

    for (i=1;(2*i)<=p->size;i=child)
    {
        child = i*2;
        if (p->arr[child+1].salary>p->arr[child].salary)
            child++;
        if (les<p->arr[child].salary)
        {   p->arr[i].id=p->arr[child].id;
            p->arr[i].salary=p->arr[child].salary;
        }
        else
            break;
    }
    p->arr[i].salary=les;
    p->arr[i].id=lec;
    return mec;
```

```
}
```

Pq.c

```c
#include <stdio.h>
#include <stdlib.h>
#include "pqadt.h"

void main()
{
    printf("enter the number of elements:");
    int n;
    scanf("%d",&n);
    struct pq *p;
    p=(struct pq*)malloc(sizeof(struct pq));
    init(p);
    struct employee *e;
    e=(struct employee*)malloc(sizeof(struct employee));
    for (int i=0;i<n;i++)
    {
        printf("enter the id: ");
        char id;
        scanf(" %c",&id);  //remeber to leave space as the previous \n is
there in buffer
        e->id=id;
        printf("enter the salary: ");
        long int sal;
        scanf("%ld",&sal);
        e->salary=sal;
        insert(p,e);
    }
    printf("\n---PRINTING--\n");
    display(p);
    printf("\n");
    printf("--1ST DELETION--\n");
    printf("HIGHEST PRIORITY : %c",delete(p));
    printf("\n---PRINTING--\n");
    display(p);
    printf("\n");
    printf("--2ND DELETION--\n");
    printf("HIGHEST PRIORITY : %c",delete(p));
    printf("\n---PRINTING--\n");
    display(p);
    printf("\n");
    printf("--3RD DELETION--\n");
```

```
    printf("HIGHEST PRIORITY : %c",delete(p));
    printf("\n---PRINTING--\n");
    display(p);
    printf("\n");
}
```

OUTPUT:

```
C:\Users\User\OneDrive\SEM3\DATA STRUCTURES\LAB\HEAP>f1.exe
enter the number of elements:7
enter the id: A
enter the salary: 15000
enter the id: K
enter the salary: 12000
enter the id: R
enter the salary: 4000
enter the id: T
enter the salary: 3500
enter the id: L
enter the salary: 4600
enter the id: P
enter the salary: 6000
enter the id: Y
enter the salary: 8600

---PRINTING--
(A,15000) (K,12000) (Y,8600) (T,3500) (L,4600) (R,4000) (P,6000)
--1ST DELETION--
HIGHEST PRIORITY : A
---PRINTING--
(K,12000) (P,6000) (Y,8600) (T,3500) (L,4600) (R,4000)
--2ND DELETION--
HIGHEST PRIORITY : K
---PRINTING--
(Y,8600) (P,6000) (R,4000) (T,3500) (L,4600)
--3RD DELETION--
HIGHEST PRIORITY : Y
---PRINTING--
(P,6000) (L,4600) (R,4000) (T,3500)
```